CSE 554 Machine Learning Systems

Lecture 7 - Performance Modeling

Baris Kasikci

BE BOUNDLESS

Class Overview

1. GPU Architecture and Programming
2. Transformers (Architecture and Implementation)
3. **Performance modeling**
4. Quantization
5. Memory Management
6. Sparsity
7. Mixture of Experts (MoE)
8. Batching
9. Speculative decoding
10. Parallelism
11. End-to-end serving
12. Agentic systems
13. Large-scale deployment

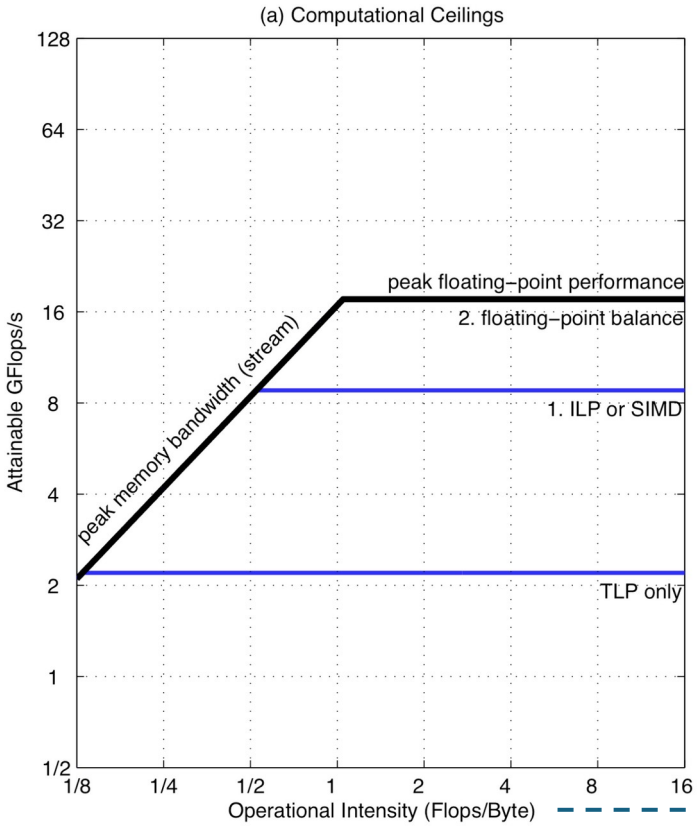
Today

- Performance Analysis
  - Roofline model
    - The first part will follow the Roofline paper and the roofline chapter from Google's scaling book
  - Detailed model of Transformer performance

The Roofline Model

Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures*

Samuel Williams, Andrew Waterman, and David Patterson
Parallel Computing Laboratory, 565 Soda Hall, U.C. Berkeley, Berkeley, CA 94720-1776, 510-642-6587
samw, waterman, pattrsn@eecs.berkeley.edu



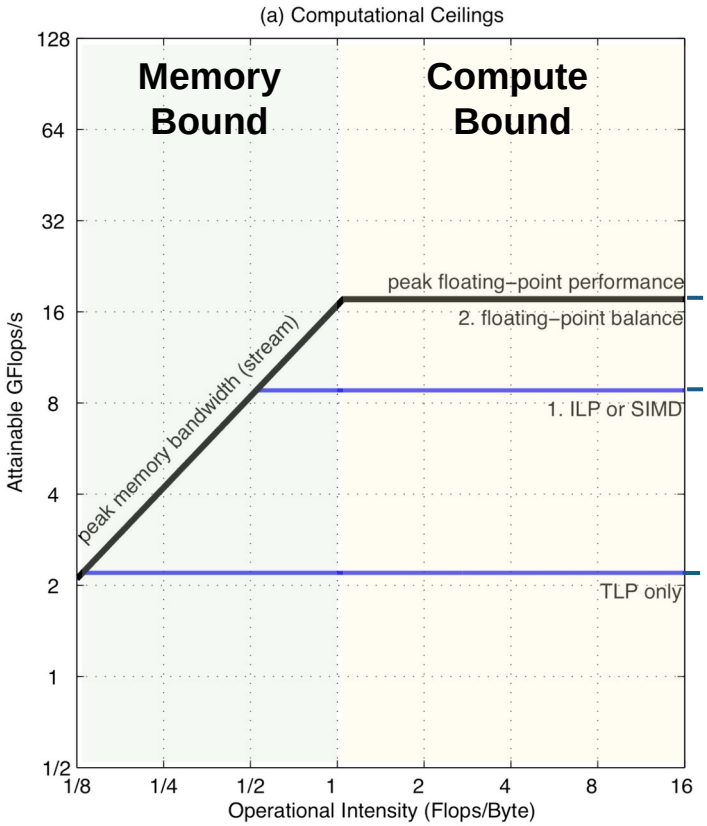
Roofline Model with Ceilings for Opteron X2

$$\textbf{Operational Intensity} = \frac{\textbf{\# of Operations}}{\textbf{\# of Bytes Moved}}$$

E.g., FLOPs/Sec

E.g., Bytes/sec (Memory or Network Bandwidth)

The Roofline Model – Computational Ceilings



Roofline

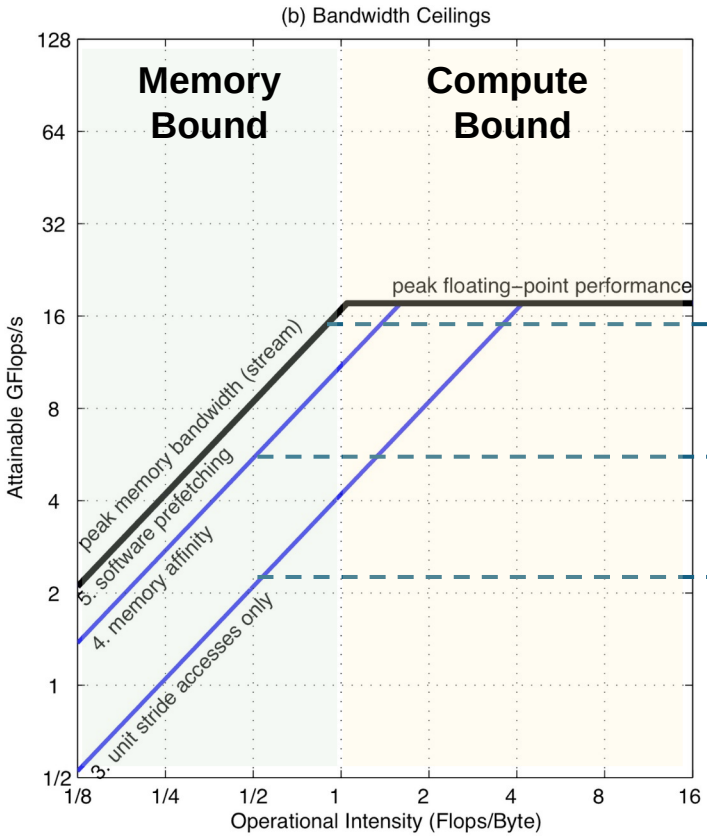
Ceiling #2

Ceiling #1

Compute optimizations: Multithreading, ILP, SIMD

Roofline Model with Ceilings for Opteron X2

The Roofline Model – Bandwidth Ceilings



Roofline Model with Ceilings for Opteron X2

Roofline

Ceiling #2

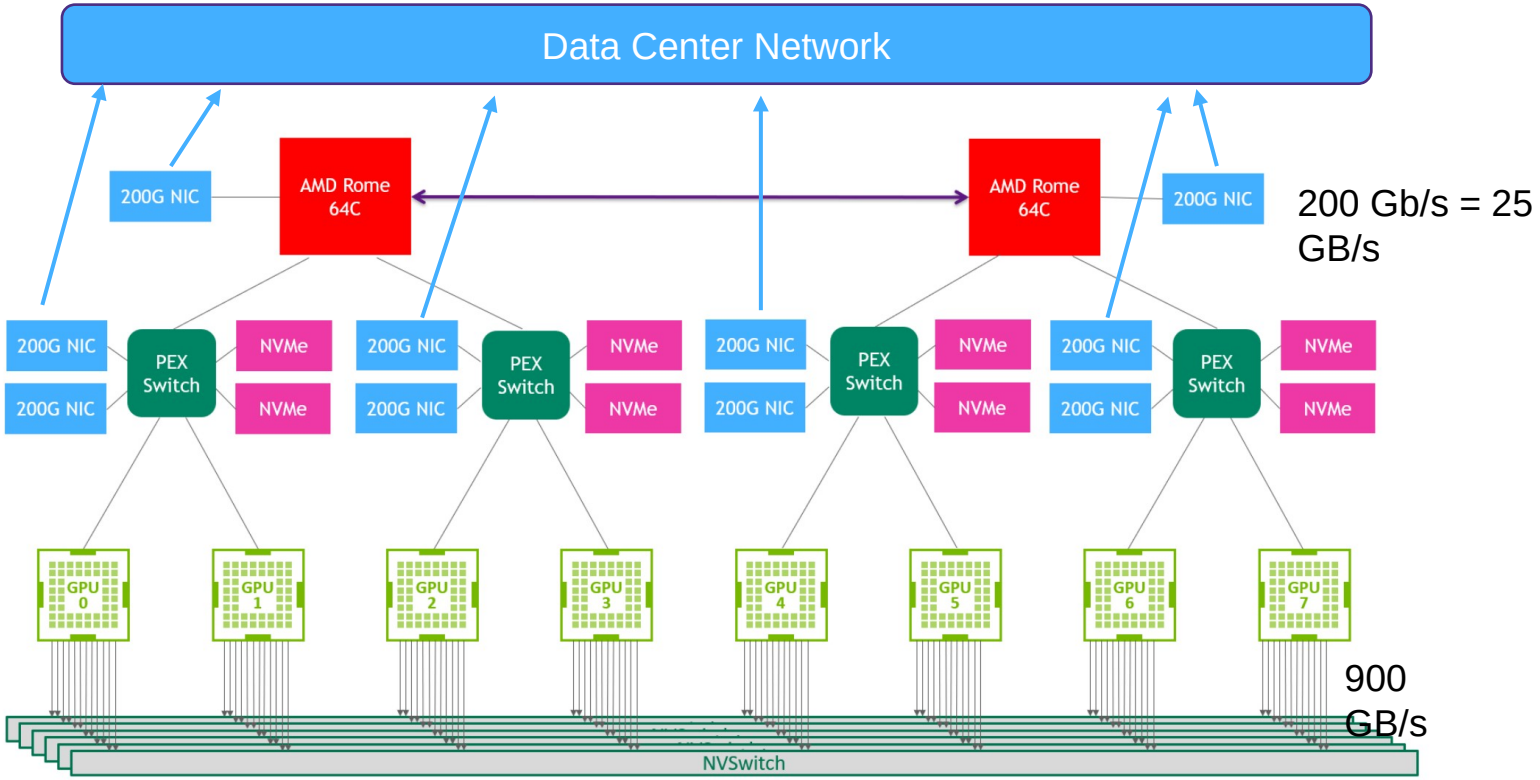
Ceiling #1

Memory optimizations:

Stride accesses (HW prefetching),

Memory affinity (NUMA effect), Software prefetching

NUMA Effect with GPUs

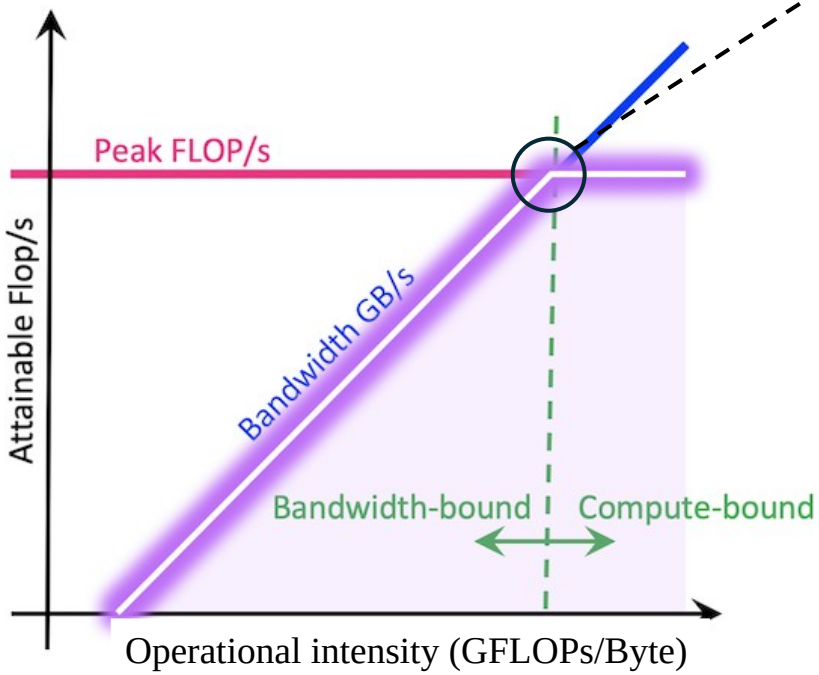


https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf

Memory vs Compute Bound

- Memory-Bound: Low Operational Intensity (OI)
  - Doing little computation per byte of data fetched
  - Spending time waiting for data rather than doing computation
- Example:
  - Accelerator with a compute capacity of 10 TFLOPs/s
  - Kernel with 1 FLOP per 10 Bytes fetched, OI: 0.1 FLOP/byte
  - Memory BW of 100 GB/s
  - Max performance: BW x OI = 100 x 10^9 x 0.1 = 10 GFLOPs/s
    - Even though the processor can do more, it can’t get the data!
- Compute-bound: High OI
  - Doing a lot of computation per byte of data fetched from memory

Critical Operational Intensity



Computation could be a GPU kernel, TPU program, CPU program etc.

Intensity(Computation) = Intensity(Accelerator)

$$\frac{\text{Computation FLOPs}}{\text{Communication Bytes}} = \frac{\text{Accelerator FLOPs/s}}{\text{Bandwidth Bytes/s}}$$

Compute-Bound

$$\frac{\text{Computation FLOPs}}{\text{Communication Bytes}} > \frac{\text{Accelerator FLOPs/s}}{\text{Bandwidth Bytes/s}}$$

Memory-Bound

$$\frac{\text{Computation FLOPs}}{\text{Communication Bytes}} < \frac{\text{Accelerator FLOPs/s}}{\text{Bandwidth Bytes/s}}$$

https://jax-ml.github.io/scaling-book/roofline/, https://docs.nersc.gov/tools/performance/roofline/

Example – NVIDIA H100

- Peak FP16 Tensor TFLOPs/s (no sparsity/with sparsity)
  - 1000/2000 TFLOPs/s
- Memory bandwidth
  - 3000 GB/s
- Critical intensity:
  - (1000*1e12) / (3000*1e9) = **333 FLOPs/Byte**
  - If the kernel has higher operational intensity than 333 FLOPS/Byte it is compute-bound, otherwise it is memory bound

Example: Dot Product with FP32

$$\underbrace{\begin{bmatrix} A_1 & A_2 & A_3 & \ldots & A_N \end{bmatrix} \begin{bmatrix} B_1 \\ B_2 \\ B_3 \\ . \\ . \\ . \\ B_N \end{bmatrix}}_{\text{Memory}} = \underbrace{\begin{bmatrix} B_1A_1 + B_2A_2 + B_3A_3 + \ldots + A_NB_N \end{bmatrix}}_{\text{Compute}}$$

Memory

2 x 4N bytes read +
4 bytes written back

Compute

N multiplications, N additions*
= 2N FLOPs

$$\textbf{Operational Intensity} = \frac{2N}{4N + 4} \approx \frac{1}{2}$$

FP32 dot product on the H100 is memory-bound

* It is actually N and not N-1, why?

Example: Matrix Multiplication with FP16

[M,N]@[N,K]→ [M,K]

$X \mathbin{@} Y \rightarrow Z$

Memory

2MN + 2NK bytes read
2MK bytes written back

Compute

2MNK FLOPs
(MNK multiplications and MNK additions)

Transformer matrix multiplications (e.g., QKV generation): N,K >> M

hidden_dim, num_qo_heads*head_dim

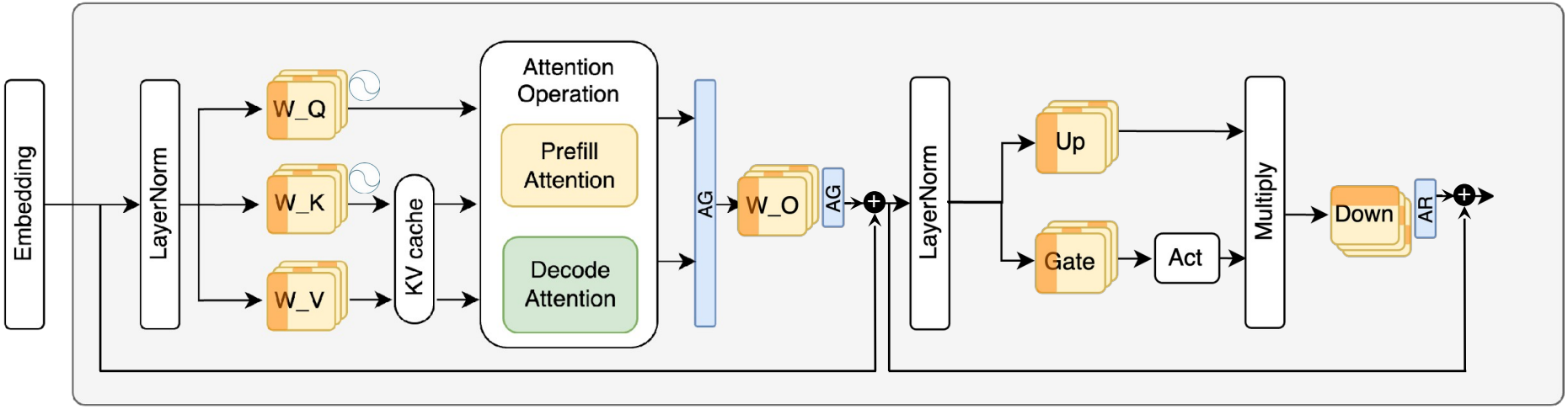
seq_len

Operational Intensity $= \dfrac{2MNK}{2MN + 2NK + 2MK} = \dfrac{2MNK}{2MN + 2NK + 2MK} \approx M$

Matrix multiplication on H100 is compute-bound if M > 333

Critical intensity for H100 from 2 slides back

Decoder-Only Transformer Architecture



Key Factors For Serving Throughput – Hardware Spec

N_{GPU}: number of GPUs

MemBW (GB/s): aggregate GPU memory bandwidth

GPU_{mem} (GB): aggregate GPU memory capacity

Compute (GFLOPS/s): aggregate GPU compute capacity

NetBW (GB/s): aggregate GPU interconnect bandwidth

Key Factors For Serving Throughput – Model Config

$\mathbf{D_{model}}$: hidden dimension size (hidden_dim)

$\mathbf{L}$: number of layers

$\mathbf{P_{model}}$: number of parameters

$\mathbf{R_{GQA}}$: group size of GQA (group_size)

$\mathbf{S_{Type}}$: Model parameters' data size in bytes (e.g., 2 for FP16)

Key Factors For Serving Throughput – User Stats

p: average number of tokens in prompts to be prefilled

d: average number of tokens in output to be decoded

---

p + d: average number of tokens in user request (sequence length)

$$\frac{\text{Throughput}_{\text{total}}}{p + d}$$ **:** per request throughput

$$d \frac{\text{Throughput}_{\text{total}}}{p + d}$$ **:** decoding throughput

Assumptions

- We are operating at the maximum possible batch size of requests
  - Necessary for high GPU utilization

Memory-Centric Execution Time Estimation*

Entire contents of the GPU memory would be loaded once during one iteration

$$T_{memory} = \frac{GPUmem}{MemBW}$$

* We assume operation at maximum batch size, which is necessary for optimal throughput

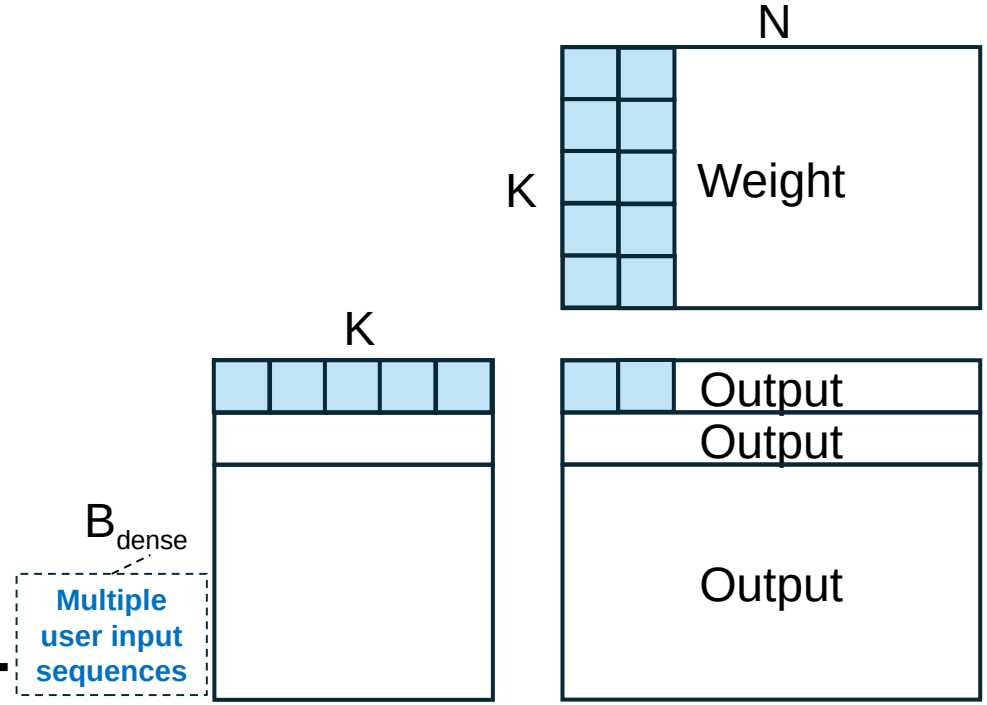
Compute-Centric Execution Time Estimation

Every GEMM compute = $2\,B_{dense}\;K\;N$

All Dense Operations

$$
\begin{array}{lllll}
 & 2\,B_{dense} & K_1 & N_1 \\
+ & 2\,B_{dense} & K_2 & N_2 \\
+ & 2\,B_{dense} & K_3 & N_3 \\
+ & \cdots\;\cdots & \cdots & \cdots \\
+ & 2\,B_{dense} & K_i & N_i \\
\hline
 & 2B_{dense} & P_{model} &
\end{array}
$$

Activations being multiplied with weights in large batches



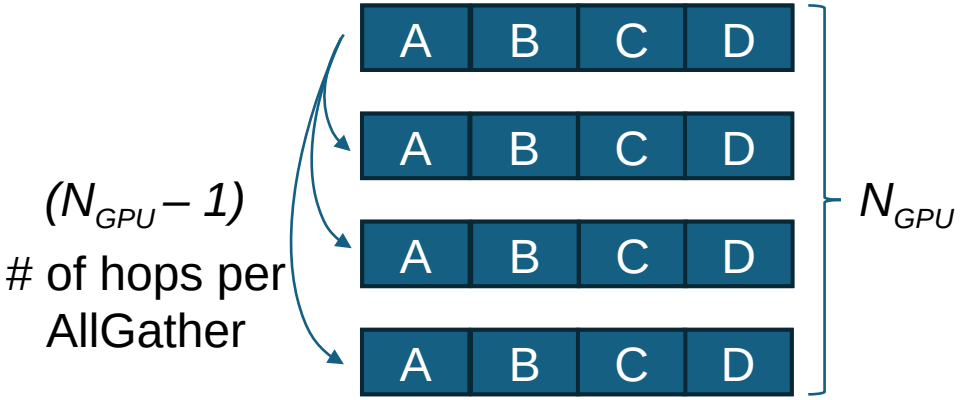
Multiple user input sequences

$$T_{compute} = \frac{2B_{dense}P_{model}}{Compute}$$

Network-Centric Execution Time Estimation

***(N_{GPU} – 1)**: # of hops*

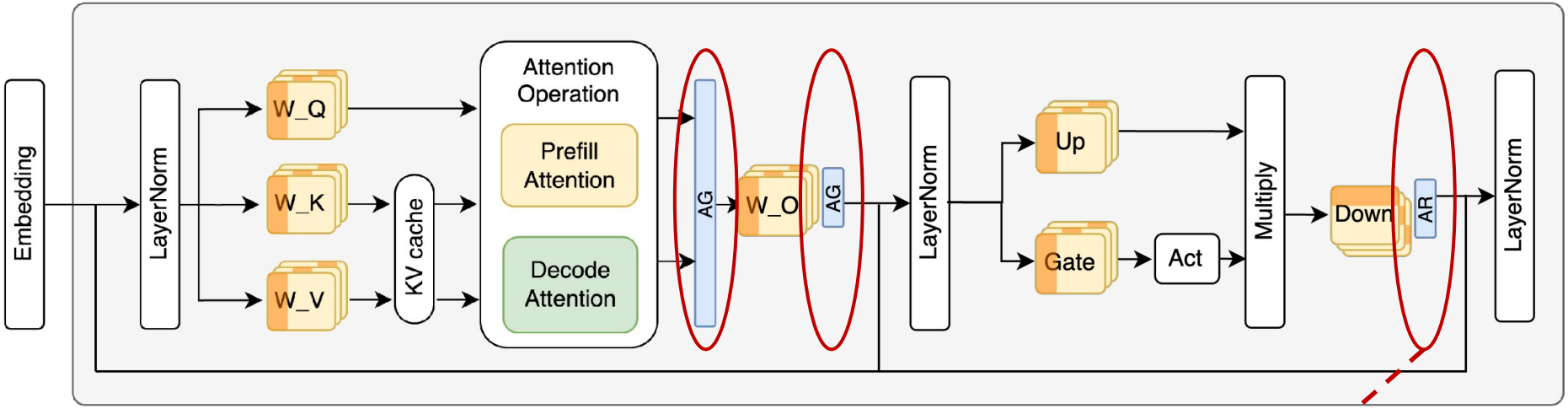
All-Gather Recap



Network-Centric Execution Time Estimation

***(N_{GPU} – 1)**: # of hops*

***4**: All-Gathers/layer*



All-Reduce ≈ 2 x All-Gather

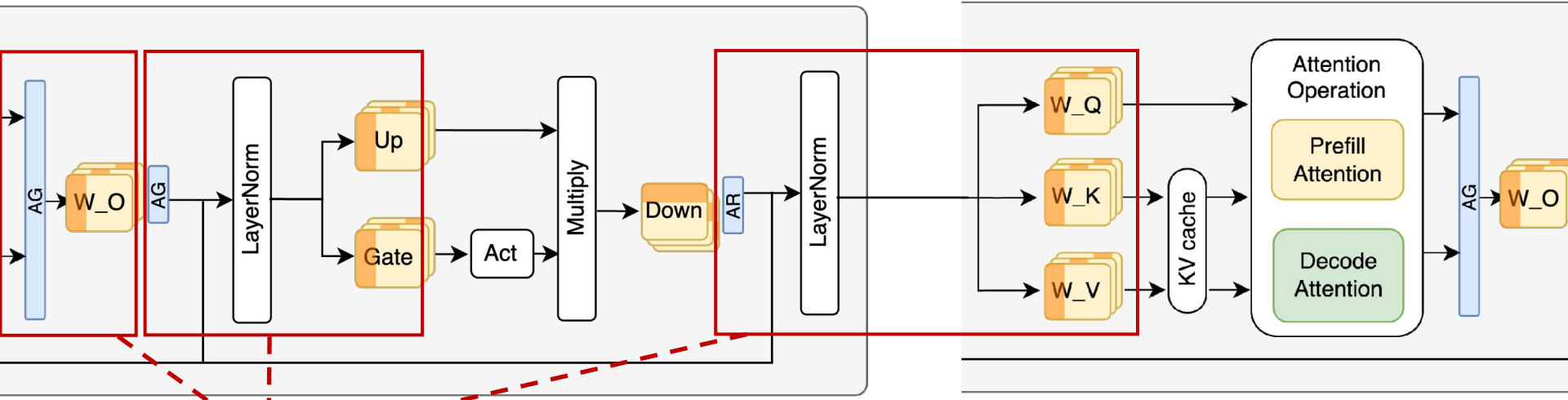
Network-Centric Execution Time Estimation

(N_{GPU} – 1): *# of hops*

4: *All-Gathers/layer*

B_{dense} x D_{model}: *shape*

S_{type}: *data type size*



$[B_{dense}, D_{model}]$ *Each element of size S_{type}*

Network-Centric Execution Time Estimation

(N_{GPU} – 1): *# of hops*

4: *All-Gathers/layer*

B_{dense} x D_{model}: *shape*

S_{type}: *data type size*

$$T_{net} = \frac{4(N_{GPU} - 1)D_{model}B_{dense}S_{type}}{NetBw} L$$

Execution Times wrt Different Resources

$$T_{compute} = \frac{2B_{dense}P_{model}}{Compute}$$

$$T_{memory} = \frac{GPUmem}{MemBW}$$

$$T_{net} = \frac{4(N_{GPU} - 1)D_{model}B_{dense}S_{type}}{NetBw}$$

Execution Times wrt Different Resources

$$T_{compute} = \frac{2B_{dense}P_{model}}{Compute}$$

$$T_{memory} = \frac{GPUmem}{MemBW}$$

$$T_{net} = \frac{4(N_{GPU} - 1)D_{model}B_{dense}S_{type}}{NetBw}$$

Compute vs Network

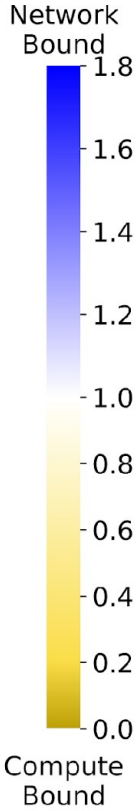
$$\frac{T_{net}}{T_{compute}} = 2(N_{GPU} - 1)\frac{D_{model}L}{P_{model}} \quad \frac{S_{type}\ Compute}{NetBw}$$

Shift to Compute Bound	Shift to Network Bound	Not Relevant
P_{model}	$Compute$	B_{dense}
$NetBW$	S_{type}	

LLM Serving is more compute-bound than network-bound

Compute vs Network - Validation

	V100	A100 (40GB)	A100 (80GB)	H100	H200	B100	B200	MI250	MI300	MI325X	Gaudi2	Gaudi3	Ada6000 (PCIE)
LLaMA-3 8B 8xGPU	0.382	0.477	0.477	1.008	1.008	0.918	1.147	0.415	1.171	1.171	1.529	1.376	2.609
Mistral 8x7B 8xGPU	0.243	0.303	0.303	0.640	0.640	0.583	0.728	0.264	0.744	0.744	0.971	0.874	1.657
LLaMA-2 70B 8xGPU	0.218	0.273	0.273	0.576	0.576	0.524	0.655	0.237	0.669	0.669	0.874	0.786	1.491
LLaMA-3 70B 8xGPU	0.218	0.273	0.273	0.576	0.576	0.524	0.655	0.237	0.669	0.669	0.874	0.786	1.491
Qwen2 72B 8xGPU	0.212	0.265	0.265	0.560	0.560	0.510	0.637	0.231	0.651	0.651	0.850	0.765	1.450
LLaMA-3 405B 8xGPUx2PP	0.119	0.148	0.148	0.314	0.314	0.285	0.357	0.129	0.364	0.364	0.476	0.428	0.812



Execution Times wrt Different Resources

$$T_{compute} = \frac{2B_{dense}P_{model}}{Compute}$$

$$T_{memory} = \frac{GPUmem}{MemBW}$$

$$T_{net} = \frac{4(N_{GPU} - 1)D_{model}B_{dense}S_{type}}{NetBw}$$

Execution Times wrt Different Resources

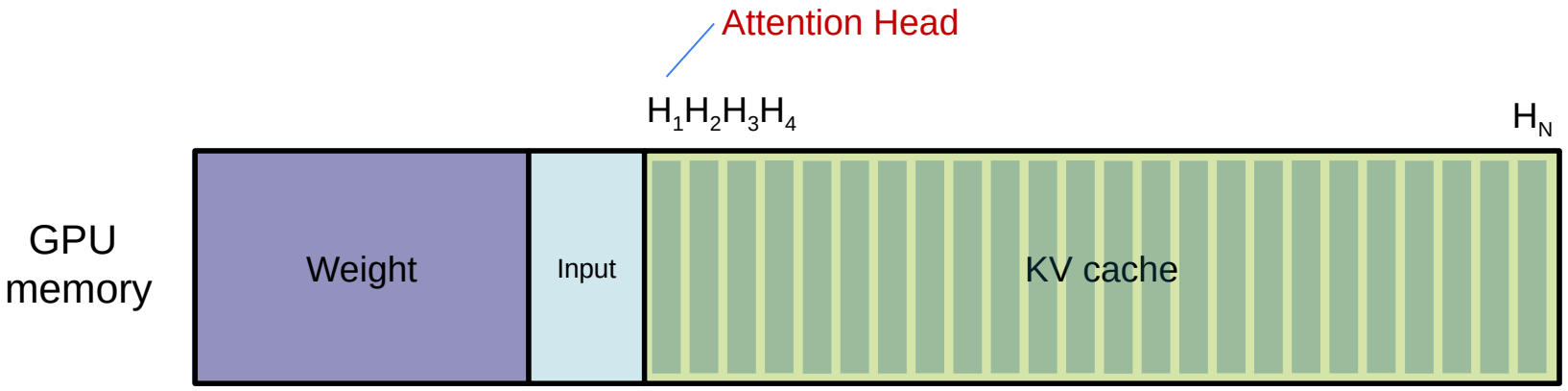
$$T_{compute} = \frac{2B_{dense}P_{model}}{Compute}$$

$$T_{memory} = \frac{GPUmem}{MemBW}$$

Compute vs Memory

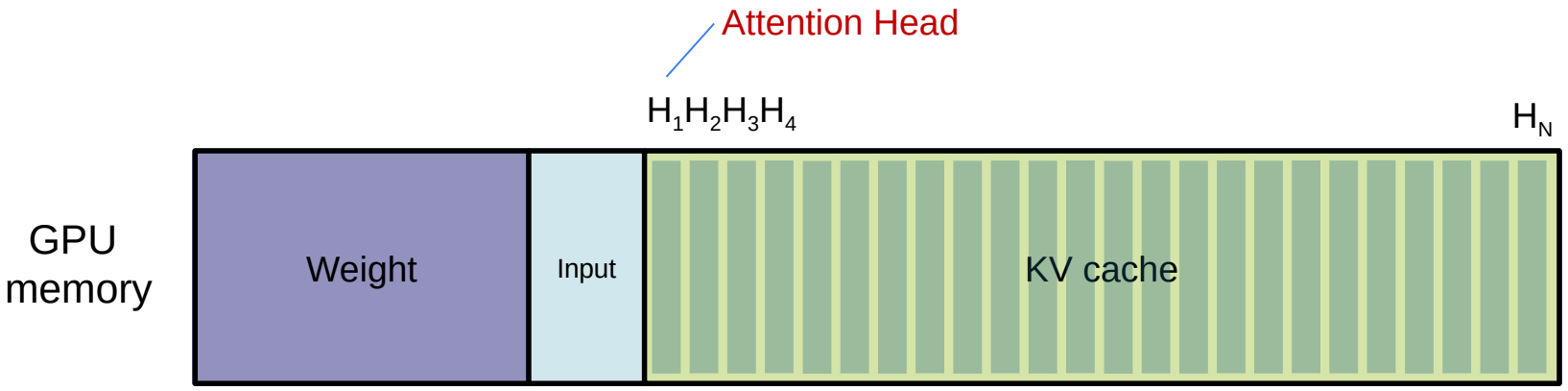
$$\frac{T_{memory}}{T_{compute}} = \frac{Compute}{MemBW} \frac{GPUmem}{2B_{dense}P_{model}}$$

Grouped Query Attention[1]



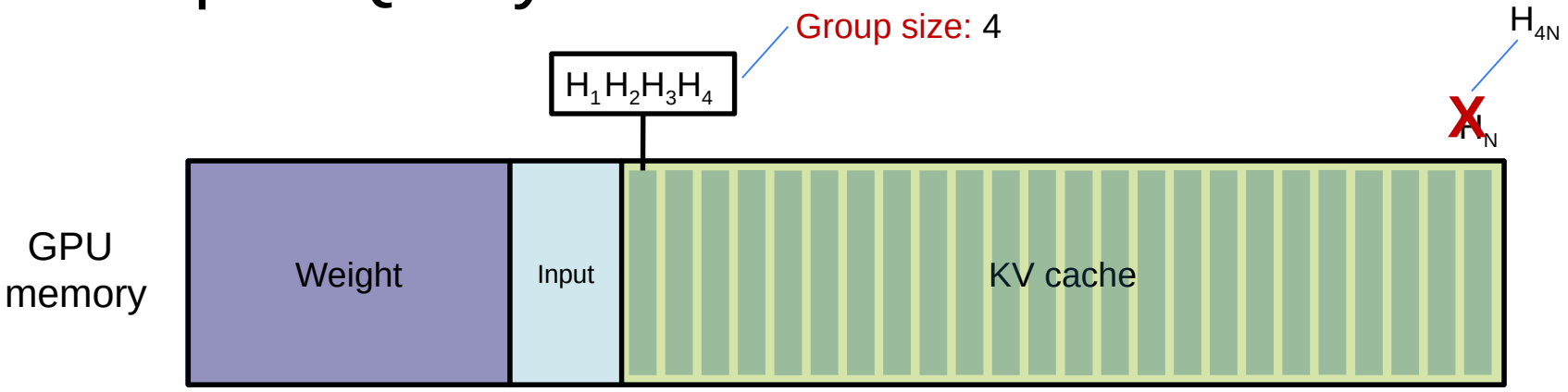
[1] GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, Sumit Sanghai

Grouped Query Attention[1]



[1] GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, Sumit Sanghai

Grouped Query Attention[1]



GQA allows increasing the batch size by a factor of the group size

[1] GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, Sumit Sanghai

Compute vs Memory

Compute

Memory

$$\frac{T_{memory}}{T_{compute}} = \frac{Compute}{MemBW} \frac{GPUmem}{2B_{dense}P_{model}}$$

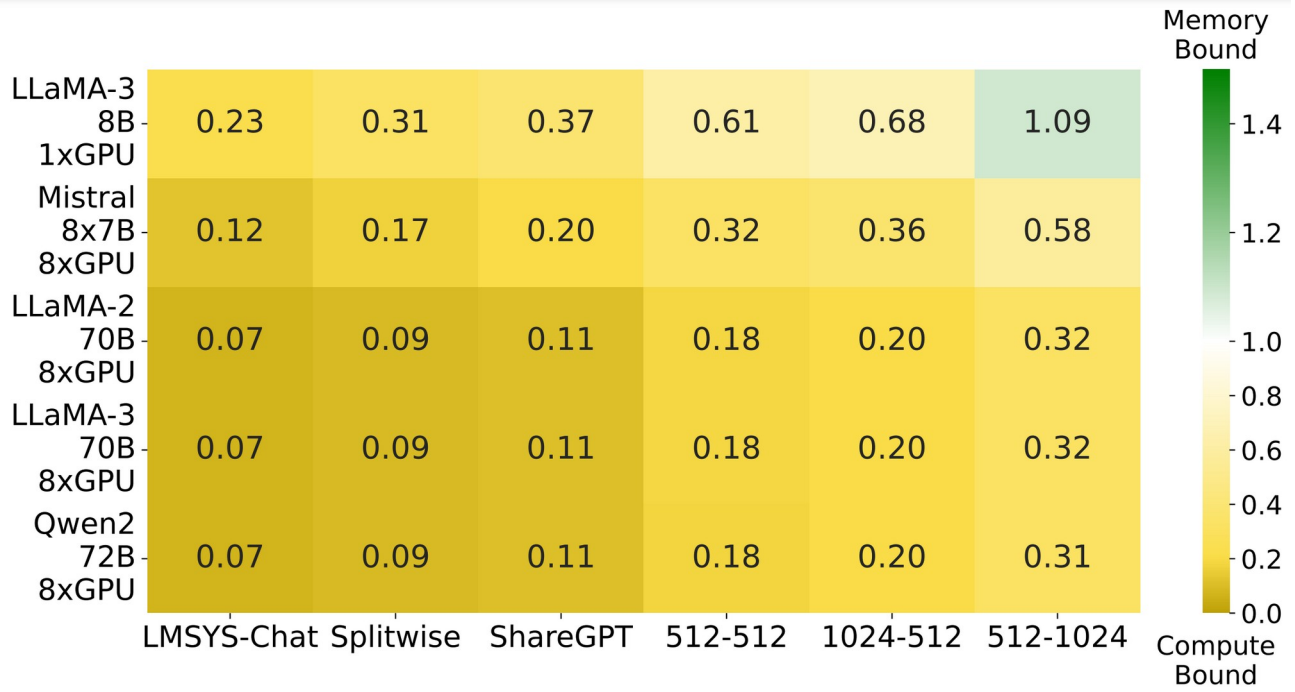
GQA allows for a large batch size

Model sizes tend to increase

Batches with large decode requests increase memory accesses

LLM serving is more compute-bound than memory-bound

Compute vs Memory - Validation



*The GPU here is A100.

High Utilization of GPU compute is the key for LLM serving

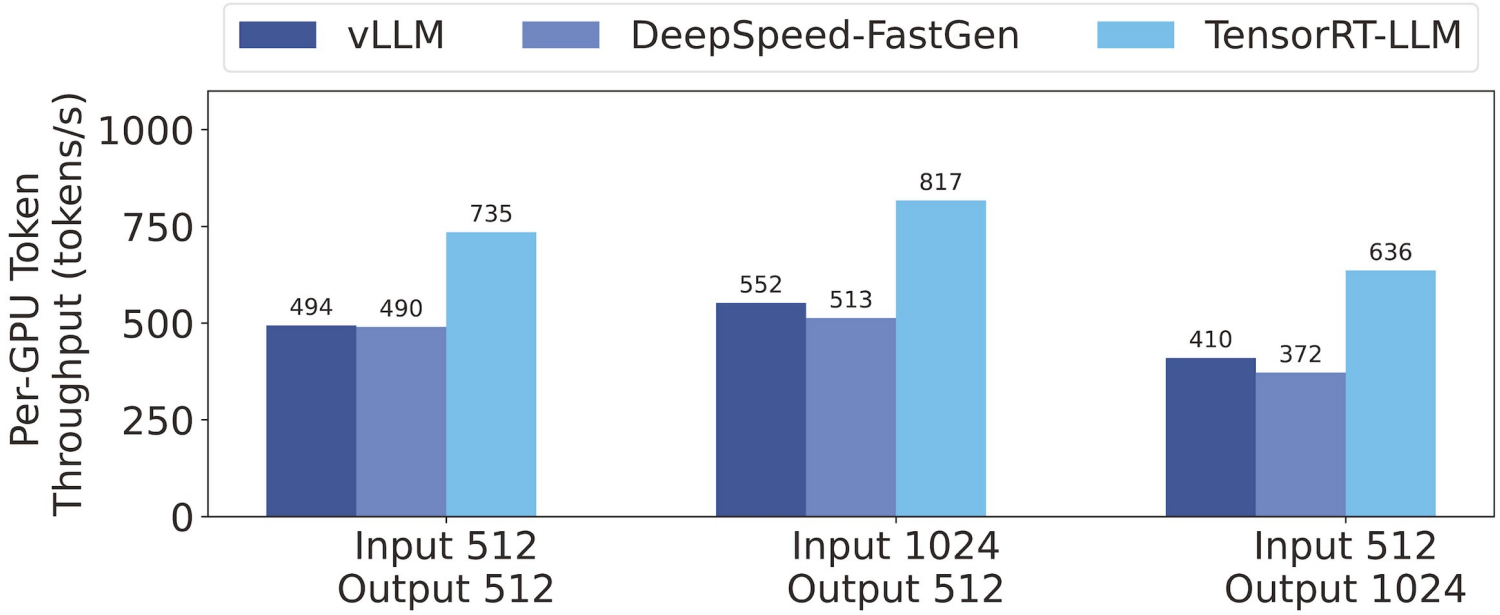
Optimal Throughput

$$Throughput = \begin{matrix} Decode \\ Throughput \\ + \\ Prefill \\ Throughpu \\ t \end{matrix} = \frac{B_{decode} + B_{prefill}}{me\ per\ Iteratic} = \frac{B_{dense}}{T_{compute}} = \frac{B_{dense}}{\frac{2B_{dense}P_{model}}{Compute}}$$

$$Throughput = \frac{Compute}{2P_{model}}$$

e.g., LLaMa 70B, A100
1857 token/s/GPU

Revisit the performance of existing frameworks

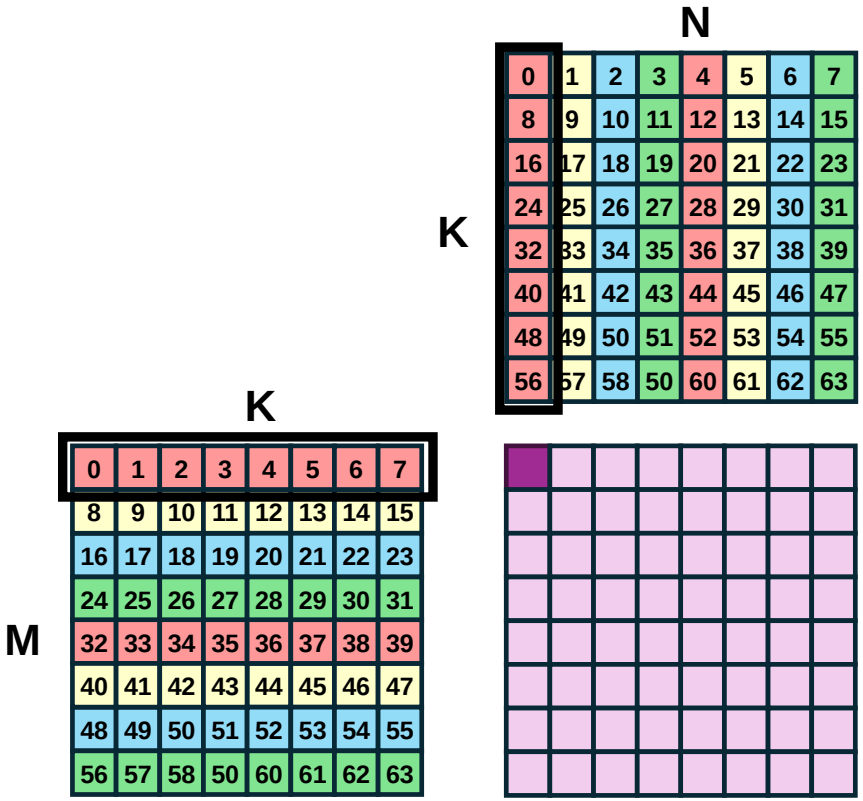


There is a large gap to optimal throughput

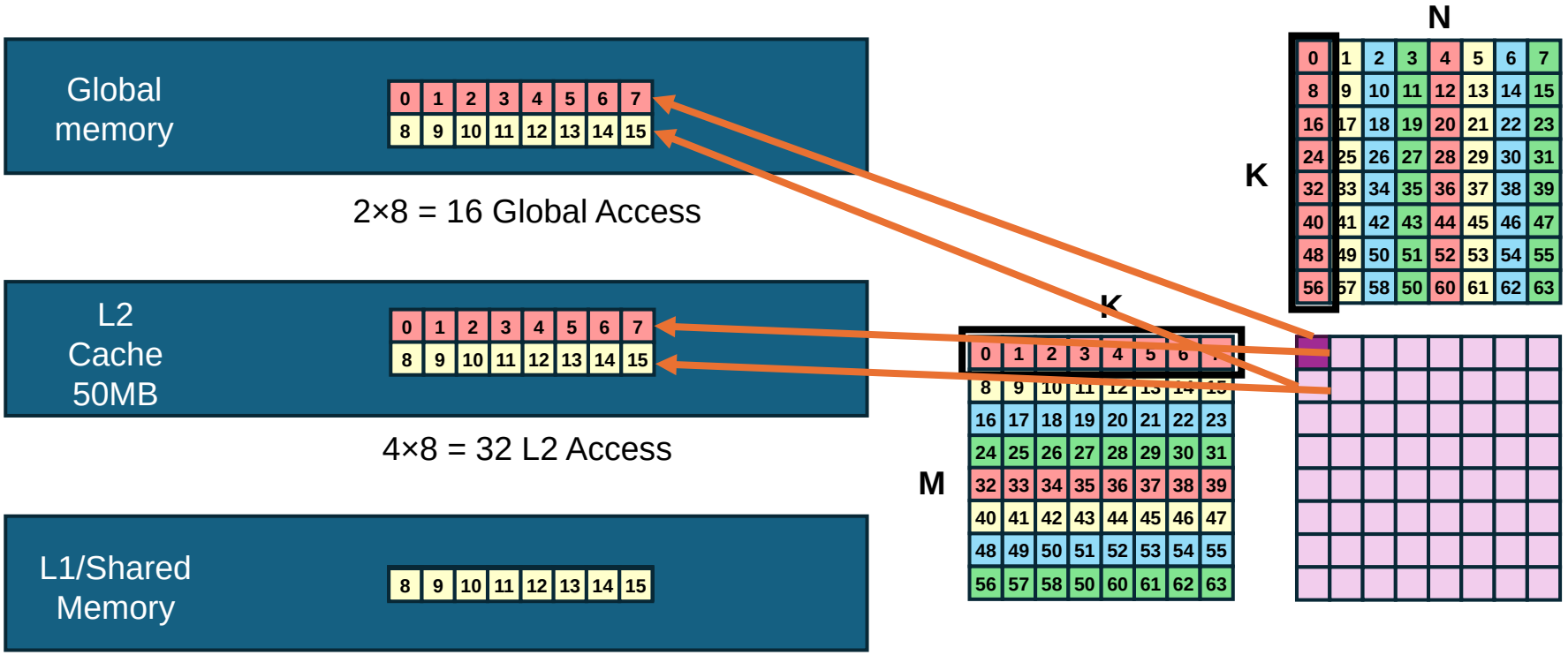
Memory Optimizations

- KV Cache
  - Recall in the transformer implementation how we recompute keys and values at every iteration for each request
    - We can instead cache them!
  - Assignment 2
- PagedAttention
- FlashAttention

Compute Optimizations for GEMM

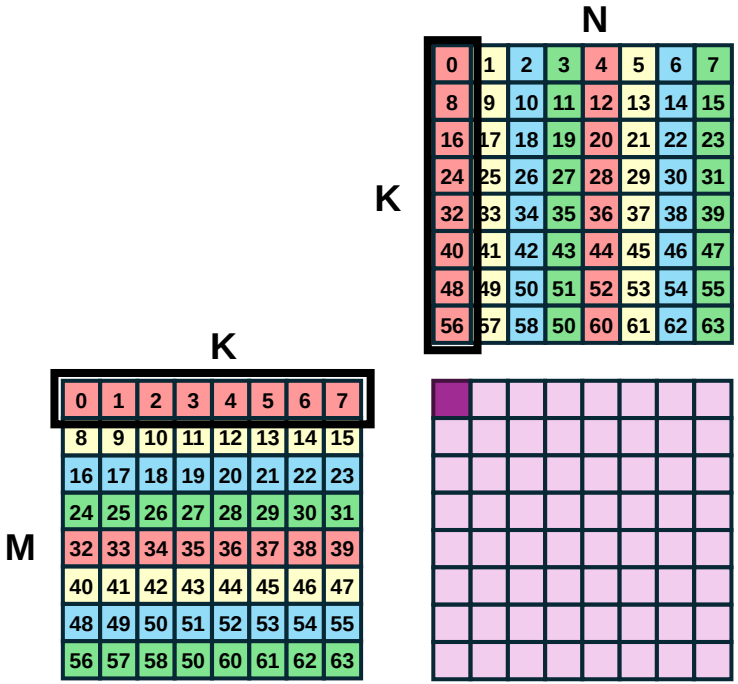


GEMM memory load pattern

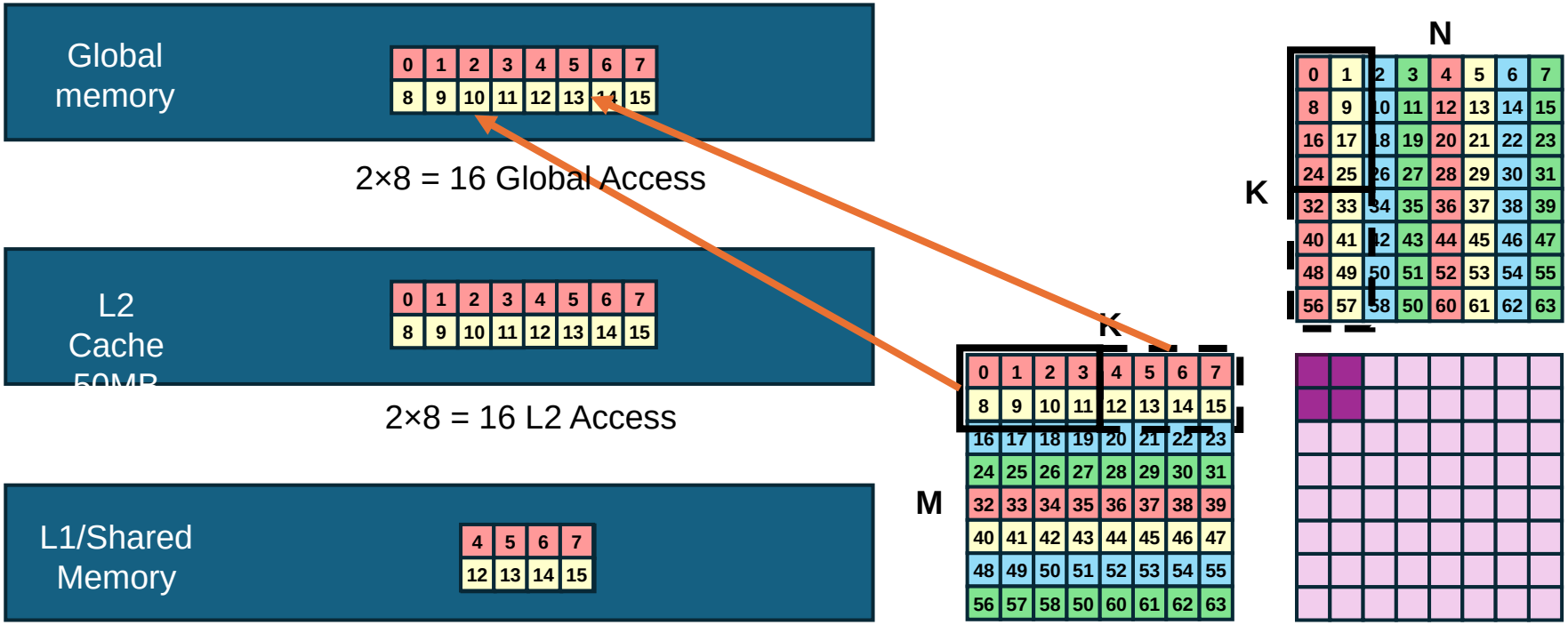


GEMM memory load

- For every output element:
  - Load one row + one column
  - 2K elements
- In total MN output elements
- Memory load = 2MNK
- However, unique load is only
  - MK + NK
- How can we reduce L2 accesses?
  - Use tiling!

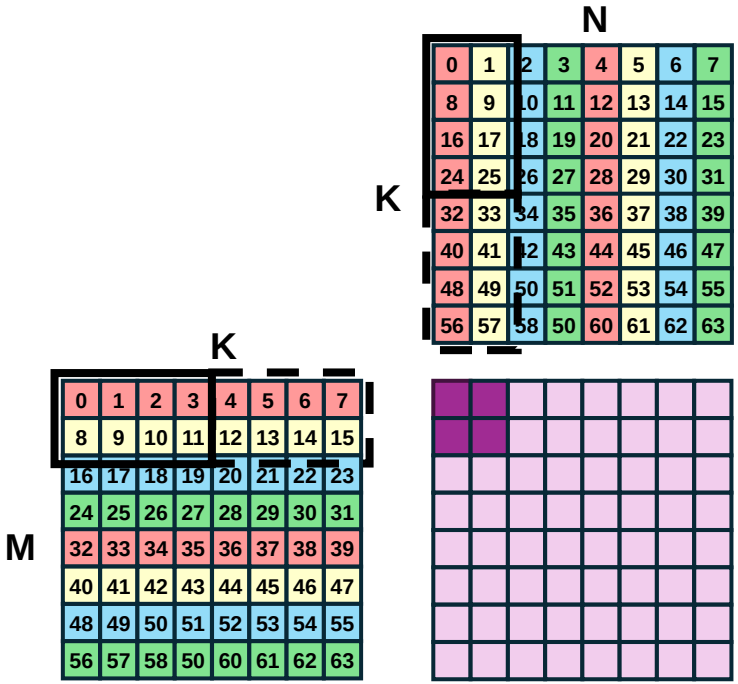


GEMM memory load pattern



GEMM Tiling

- Load by Tiles
  - Input: TILE_M * K
  - Weight: K * TILE_N
  - Output: TILE_M * TILE_N
- Memory Load
  - For every output tile
    - Load TILE_M rows and TILE_N columns
    - (TILE_M + TILE_N) K
  - MN/TILE_M/TILE_N: total output tiles
  - Total memory load
  - $L = \frac{Tile_M + Tile_N}{Tile_M Tile_N} MNK$



L2 Access reduced by a factor of tile dimension (Assuming TileM = TileN)

Tensor Cores – Specialized for small GEMMs

What is a tensor core?

- A special hardware unit that performs a small shape GEMM
- A warp (32 threads) collectively uses the tensor core

PTX	Data Types (A * B + C)	Shape	Speedup on NVIDIA A100 (vs F32 CUDA cores)	Speedup on Turing* (vs F32 Cores)	Speedup on Volta* (vs F32 Cores)
mma.sync.m16n8k16 mma.sync.m16n8k8	F16 * F16 + F16 F16 * F16 + F32 BF16 * BF16 + F32	16-by-8-by-16 16-by-8-by-8	16x	8x	8x
mma.sync.m16n8k8	TF32 * TF32 + F32	16-by-8-by-8	8x	N/A	N/A
mma.sync.m8n8k4	F64 * F64 + F64	8-by-8-by-4	2x	N/A	N/A
mma.sync.m16n8k32 mma.sync.m8n8k16	S8 * S8 + S32	16-by-8-by-32 8-by-8-by-16	32x	16x	N/A
mma.sync.m16n8k64	S4 * S4 + S32	16-by-8-by-64	64x	32x	N/A
mma.sync.m16n8k256	B1 ^ B1 + S32	16-by-8-by-256	256x	128x	N/A

https://developer.download.nvidia.com/video/gputechconf/gtc/2020/presentations/s21745-developing-cuda-kernels-to-push-tensor-cores-to-the-absolute-limit-on-nvidia-a100.pdf